

```
#include <iostream>
#include <cstdlib>
#include "Calc.h"
int main() {
    Calc C1;
    return 0;
}
```

기존의 c와 다르게 c++에서는 객체를 사용하기 때문에 이에 맞춰서 객체에 포커스를 맞춰서 코드를 짜봤다.

파일을 main.cpp 하나에만 작성하는 것이 아니라 calc.h, calc.cpp, main.cpp로 헤더, 소스, 메인파일 형식으로 작성하였다.

헤더, 소스파일을 나누는 것이 본 과제의 의도라고 생각하여서 3파일에 나누어서 코드를 작성하였다.

이를 통해 선언부, 동작부를 나누어서 관리해 코드에서 오류가 뜨면 어디가 문제인지를 파악하기 편하였다.

생성자, 소멸자가 c에서는 없는 개념이라 이것을 익혀 보기 위해서 적극활용하였다.

이번과제에서는 순차구조로도 프로그램이 작동하기 때문에 생성자에 함수를 넣어서 객체를 선언하기만 하면 저절로 프로그램이 돌아가게 만들어 보았다.

아마 생성자는 로봇쪽으로 사용을 하게 된다면 센서나 모듈의 init처럼 초기화를 하는데 사용할 것 같다.

소멸자는 객체가 끝나게 되면 자동으로 실행되는 함수로 과제 코드에서는 따로 동작시키진 않았다. 하지만 main함수에서 return 0;을 하면 자동으로 실행된다고 하여서 동적할당의 메모리 해제를 소멸자함수로 사용하였다.

```

day1 > hw1 > C Calc.h > Calc
1  ▾ #ifndef CALC_H
2    #define CALC_H
3  ▾ class Calc {
4      private:
5          int *arr= nullptr; //원소를 저장할 배열
6          int len;           //배열의 길이(동적할당)
7          int max;           //원소의 최댓값
8          int min;           //원소의 최솟값
9          int sum=0;         //원소 전체합
10     public:
11         Calc();
12         ~Calc();
13         void input_len();    //배열이 길이 입력
14         void remake_arr();   //배열 동적할당하기
15         void input_element(); //원소 입력받기
16         void Calc_main();    //max, min, sum, avg 구하기
17         void print_result(); //출력하기
18     };
19
20 #endif

```

<객체 선언부>

```

● 몇개의 원소를 할당하시겠습니까?: 7
1번째 원소를 입력해주세요: 31
2번째 원소를 입력해주세요: 22
3번째 원소를 입력해주세요: 75
4번째 원소를 입력해주세요: 2
5번째 원소를 입력해주세요: 92
6번째 원소를 입력해주세요: 65
7번째 원소를 입력해주세요: 34
최댓값: 92
최솟값: 2
전체합: 321
평균: 45.8571
○ PS C:\Users\dhcho\OneDrive - 광운대학교\문서\Robit_intelligence>

```

<실행결과>

선언부와 실행결과를 같이 보면서 이야기해 보자면 먼저 `int *arr=nullptr;` 로 배열을 포인터로 선언하였다.

터미널에서 원소의 개수를 입력받아 `len` 변수에 저장하였다.

`remake_arr` 함수에서 동적할당을 이용하여 `arr[len]`의 형태의 배열을 만들었다.

이후 `len` 변수 값만큼 `for`문을 이용하여 `arr[i]`에 원소값을 입력받았다.

Calc_main에서 for문을 이용하여 조건문으로 비교하여 max, min, sum을 구하였다.

print_result에서 max, min, sum을 출력하고 , 평균은 (float)sum/len으로 출력하였다.